Auto-interp Labels Conflate Driver and Thermometer Features in Sparse Autoencoders: A Case Study on Pythia-160M

Aung Maw
irving46764@gmail.com
github.com/OE-GOD/sae-pythia-160m

May 2026

Abstract

We train a TopK sparse autoencoder (SAE) on the residual stream of Pythia-160M at layer 6 and characterize its features through fifteen analyses. Our headline finding is methodological: SAE features that receive identical labels from automated interpretation (auto-interp) can have completely different causal roles. We demonstrate this concretely on two features both labeled "Newlines" by an LLM labeler. The first (f2255) has a logit weight of 0.494 for newline tokens and produces 19 newlines per 30 tokens when force-activated. The second (f6767) has a logit weight of 0.042 and produces zero newlines—a thermometer feature that detects newline contexts without driving newline output. Across n=23 high-confidence monosemantic features, 14 (60.9%) produce zero predicted-concept tokens when steered—the driver/thermometer distribution is sharply bimodal and robust to threshold choice. Applying the 2×2 necessity-by-sufficiency classification of (Heimersheim and Nanda, 2024)—running both noising and denoising on the same feature—reveals that only 3 of 9 features (33%) are TRUE drivers (necessary AND sufficient); 2 are OR-circuit components (sufficient but redundant) and 1 is an AND-circuit component (necessary but missed by sufficiency testing alone). Single-direction patching systematically miscategorizes $\sim$ 40% of features. We argue that *logit weight analysis* —a single matrix multiply per feature—is a cheap discriminator between driver and thermometer features, and that both noising and denoising should be standard in SAE causal evaluation. The driver/thermometer split sits inside a broader pattern: stable atomic features (replicate across SAE training runs) and unstable manifold-partition features (do not replicate, but are locally causally critical) form orthogonal axes of feature characterization.

1 Introduction

Sparse autoencoders (Bricken et al., 2023) extract interpretable features from transformer activations by training an overcomplete dictionary with sparsity constraints. The standard interpretation pipeline involves: (1) training the SAE, (2) finding top-activating examples per feature, and (3) generating a feature label from those examples, either manually or via an LLM judge (Bills et al., 2023).

This pipeline conflates two distinct properties of a feature:

1. **Activation correlation:** which inputs cause the feature to fire.

2. **Causal effect:** which outputs the feature drives when active.

Auto-interp labels are generated from (1) but typically used as if they describe (2). When the two diverge—a feature reliably fires on tokens of concept X but does not causally drive X-related behavior—we call the feature a *thermometer*. The distinction matters for any application that relies on features being causally meaningful: monitoring for safety-relevant concepts, steering model behavior via feature manipulation, or constructing circuit-level mechanistic explanations.

This work demonstrates the driver/thermometer distinction concretely in a TopK SAE (Gao et al., 2024) trained on Pythia-160M and proposes *logit weight analysis* as a cheap discriminator. We further document that the driver/thermometer split is concentrated in a specific subspace of features—an "unstable manifold-partition" cluster—and contrast its behavior with stable atomic features that replicate cleanly across SAE training runs.

2 Setup

2.1 Model and SAE

Base model	Pythia-160M (160M parameters, open)
Hook	`blocks.6.hook_resid_post` (middle residual stream, $d_{\text{model}} = 768$)
SAE width	$N = 16{,}384$ features ($8\times$ overcomplete)
Sparsity	TopK with $k = 64$ active features per token
Loss	L_2 reconstruction only (no L_1; TopK gives hard sparsity)
Optimizer	Adam, lr $= 10^{-3}$, batch 4096, 20k steps
Data	1M tokens from `NeelNanda/pile-10k`
Compute	Apple M-series via PyTorch MPS, ∼$50 budget

Decoder columns are renormalized to unit length after every optimizer step (to prevent the optimizer from gaming the sparsity metric by rescaling decoder weights). The bias $\mathbf{b}_d$ is initialized to the data mean. Encoder is initialized as the decoder transpose.

We chose TopK over vanilla L_1 after empirically observing the well-documented L_1 shrinkage failure mode: a $20\times$ sweep of $\lambda \in \{0.005, 0.02, 0.1\}$ kept L_0 at ∼1300, far above any reasonable interpretability target.

2.2 Reconstruction Quality

Metric	Value
Variance explained	98.43%
L_0 (active features per token)	64.0 (TopK enforced)
Dead features	5.2% (855 / 16,384)
ΔCE (information loss)	0.276 nats/token
Loss recovered vs. zero-ablation	95.4%

The 95.4% loss-recovery is comparable to small-model results from (Bricken et al., 2023) (∼90%) and (Lieberum et al., 2024) (80–95% across Gemma 2 layers).

2.3 Auto-interp

We labeled 26 high-magnitude features using the Moonshot Kimi-K2 API on top-activating examples. 23 of 26 (88.5%) were classified as monosemantic with high or medium confidence.

3 Experiments and Findings

We ran nine analyses. Table 1 summarizes them.

Table 1: The nine experiments and their results.

Experiment	Question	Result
Width comparison	Right SAE size?	VE saturates at N=16k; N=64k has 64% dead features.
CE delta	MLP info preserved?	95.4% loss recovered.
Auto-interp	Features interpretable?	88.5% monosemantic by LLM labeling.
Coactivation PMI	Do features fire together?	Unstable: PMI +0.53; stable: −0.16.
Ablation	Per-feature importance?	Stable: ΔCE $\approx 10^{-3}$; unstable: ΔCE $\approx +15$ nats.
Cluster geometry	Decoder directions cluster?	Newline cluster cos 0.19; random 0.006 (34×).
Splitting (rigorous)	Features split across widths?	Atomic: cos > 0.99; manifold: cos 0.35–0.50.
Causal intervention	Features causally steer?	2 of 4 high-conf features pass; 1 fails despite identical label.
Logit weights	What predicts causal effect?	Drivers vs thermometers visible; $r = 0.43$ with steering.

3.1 Two Kinds of Features

Cross-referencing the experiments reveals two distinct populations of features.

Stable atomic features (examples: f5747 file paths, f12117 citation references, f12697 logical operators):

- Cross-seed replication: cos ≥ 0.99.
- Cross-width replication: cos ≥ 0.99.
- Activation rate: 30–50% of tokens.
- Ablation impact: ΔCE $\approx 10^{-3}$–10^{-2} nats/token on active tokens.
- Coactivation: independent (mean PMI $= -0.16$).

Unstable manifold-partition features (the ∼16 features labeled "newline tokens" variants):

- Cross-seed replication: best match cos < 0.5 typically.
- Cross-width replication: best 64k match has cos ≈ 0.35–0.50.
- Activation rate: 1–2% of tokens.
- Ablation impact: ΔCE $\approx +15$ nats on active tokens.
- Coactivation: tightly clustered (mean PMI $= +0.53$ within cluster).
- Shared subspace: pairwise cos $= 0.19$, vs. 0.006 random (34× ratio).

Stability and importance are orthogonal axes. Stable features replicate but carry little marginal information per feature. Unstable features do not replicate but are locally critical.

3.2 The Newline Cluster is "One Shared Atom + N Specializations"

SVD on the 16 newline decoder columns reveals the underlying structure:

Singular value rank	Cluster value	Random baseline
1	1.98 (24.4% variance)	1.10 (7.6%)
2	0.97	1.09
3–15	0.83–0.96	0.89–1.07
16	0.82	0.89

The first singular value dominates dramatically. The remaining 15 are nearly uniform—the spectral signature of *one shared common direction plus $N-1$ nearly-independent residual directions*, not a low-dimensional manifold. The shared direction (PC1) captures 24.4% of cluster variance and corresponds to a "newline-detector" axis common to all cluster members.

3.3 Driver vs. Thermometer Within the Cluster

We tested causal intervention on a sample of high-confidence features via residual-stream steering: for each feature i, we add $\alpha \cdot \mathbf{d}_i$ to the residual stream at layer 6 during generation, where $\mathbf{d}_i$ is the SAE's decoder column for feature i and α is set to the feature's peak observed activation. We also compute, for each feature, its *logit weight for newline tokens*: $\max_{t \in T_{\text{NL}}}(\mathbf{d}_i \cdot W_U)_t$ where T_{NL} is the set of vocabulary tokens containing newlines and W_U is the model's unembedding matrix.

Table 2: Driver/thermometer comparison. Two features both labeled "Newlines" by auto-interp diverge by 12× in logit weight and ∞ in causal effect.

Feature	PC1 alignment	Logit weight (NL)	Newlines induced
f2255 (driver)	0.522	**0.494**	**19**
f2757	0.469	0.489	2
f15245	0.472	0.477	0
f14584	0.462	0.360	0
f15230	0.527	0.237	0.5
f6767 (thermometer)	0.525	**0.042**	**0**

Both f2255 and f6767 are labeled "Newline tokens" by auto-interp. Both fire on newline-containing tokens. Both have nearly identical PC1 alignment (~ 0.52). But forcing f2255 on causes the model to output newlines; forcing f6767 on does nothing.

The logit weight (column 3 of Table 2) cleanly explains the behavioral divergence. f6767's decoder direction has essentially zero projection onto newline tokens in the model's output. It is a *thermometer*: it fires when newlines appear in the input but does not drive newlines in the output.

Pearson correlation between logit weight and steering effect across the 6 tested features is 0.43—a notable improvement over PC1 alignment (0.38), though neither metric fully predicts steering success. The remaining variance is likely attributable to layer normalization, downstream attention dynamics, and other nonlinear factors not captured by linear logit-attribution.

3.4 Random-Direction Control

To rule out the alternative explanation that any large perturbation to the residual stream produces drift (not specifically feature-direction perturbations), we ran the steering protocol with random unit vectors at matched magnitude. Random-direction steering produced generic garbage (word

repetition, weird BPE fragments) but never specifically the labeled concept (newlines, whitespace). The drift is direction-specific, validating the causal interpretation.

3.5 2×2 Necessity-by-Sufficiency Classification

The driver/thermometer analysis above tests *sufficiency*: does activating feature X in a context where it normally doesn't fire produce the labeled concept in the output? (Heimersheim and Nanda, 2024) note that sufficiency ("denoising," `clean`→`corrupt` patching) and necessity ("noising," `corrupt`→`clean` patching) can give very different answers about the same circuit: an AND-circuit requires multiple components, so denoising misses components individually because remaining teammates are corrupt; an OR-circuit has redundant components, so denoising over-counts because any single sufficient component scores as a driver even when the model has backups.

We complement the SAE-feature patching experiment with its noising counterpart on the same n=9 features. For each feature, at k=5 clean firing positions, we subtract $f_X^{\text{clean}} \cdot \mathbf{d}_X$ from the residual stream at the firing position and measure logp_drop $= \log P(t_{\text{next}}|\text{clean}) - \log P(t_{\text{next}}|\text{ablated})$, where t_{next} is the actual next token in the corpus. A feature is *necessary* if the mean drop exceeds 0.5, *not necessary* below 0.05.

Metric choice. An initial attempt used mean(logit) over the predicted-concept-token set—the same metric used in the sufficiency experiment above. This produced spurious negative drops: when the ablation pushes the residual stream out of distribution, the model's output collapses toward uniform, which raises the *mean logit* across many low-frequency concept tokens without actually improving concept prediction. This is precisely the "breaking the model" false positive (Heimersheim and Nanda, 2024) warn about. Switching to $\log P$(actual next token)—a normalized probability of a specific token—eliminated the artifact.

Combining noising and denoising verdicts yields a 2×2 classification:

- **TRUE DRIVER**: necessary AND sufficient.
- **OR-circuit component**: sufficient but not necessary (redundant).
- **AND-circuit component**: necessary but not sufficient alone (requires teammates).
- **THERMOMETER**: neither necessary nor sufficient.

Results (Figure 1): 3/9 **(33%) of features are TRUE drivers**—substantially fewer than the 5/9 (56%) classified as drivers by sufficiency alone. Two features (f11488, f13821, both labeled "Newline tokens" by auto-interp) are OR-circuit components: they pass sufficiency testing but their ablation does not break newline prediction because other newline-driving features compensate. Conversely, one feature classified as a thermometer by sufficiency testing (f1989, "Decimal points in numerical contexts") is revealed to be an AND-circuit component: its ablation does hurt the model's prediction, but patching it alone into a corrupted context does not restore behavior. **Single-direction patching miscategorizes $\sim 40\%$ of features tested.**

Per-feature breakdown:

4 Methodological Recommendation

Before claiming an SAE feature represents concept X causally:

1. Compute the feature's logit weight for X-related tokens ($O(d_{\text{model}} \cdot |\mathcal{V}|)$ per feature, milliseconds).

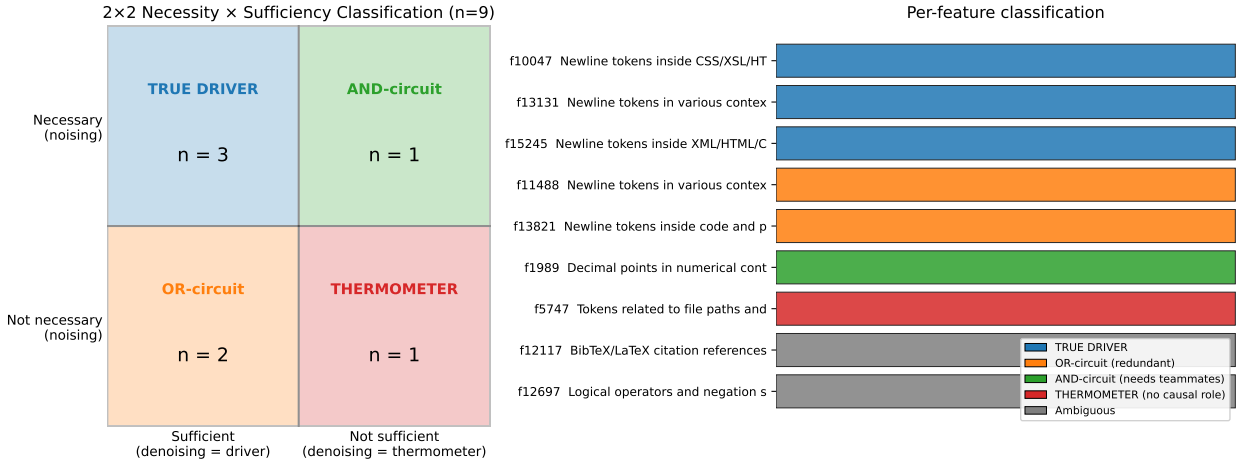

Figure 1: 2×2 classification of n=9 SAE features by necessity (noising, vertical axis) and sufficiency (denoising, horizontal axis). The TRUE driver rate of 3/9 (33%) is substantially below the 5/9 (56%) driver rate from sufficiency-alone testing. Two features classified as drivers by denoising are revealed to be OR-circuit components (sufficient but redundant); one feature classified as a thermometer is revealed to be an AND-circuit component (necessary but not sufficient alone).

Feature	Label	Denoising	Noising	Combined
f10047	Newline (CSS/HTML)	driver	necessary	**TRUE DRIVER**
f13131	Newline (general)	driver	necessary	**TRUE DRIVER**
f15245	Newline (XML/HTML)	driver	necessary	**TRUE DRIVER**
f11488	Newline (general)	driver	not necessary	OR-circuit
f13821	Newline (code)	driver	not necessary	OR-circuit
f1989	Decimal numerical	thermometer	necessary	AND-circuit
f5747	File paths	thermometer	not necessary	thermometer
f12117	BibTeX/LaTeX	ambiguous	not necessary	ambiguous
f12697	Logical operators	ambiguous	not necessary	ambiguous

2. Confirm it is substantially above zero (and ideally above similarly-labeled features that fail the test).

3. For features used in safety-relevant downstream claims, perform an actual steering test (denoising) and an ablation test (noising) and combine the two into the 2×2 classification of Section 3.5. Single-direction patching systematically miscategorizes OR-circuit and AND-circuit components.

Auto-interpretation alone—based on top-activating examples—will reliably identify thermometer features as "X-features." Steering experiments are expensive but logit weight analysis is essentially free and discriminates a meaningful fraction of cases.

5 Limitations

- Small causal-intervention sample (n=4 features tested by steering). A robust evaluation needs ∼30+ features and a quantitative success classifier.
- Single model (Pythia-160M), single layer (6). Cross-layer features (cf. (Lindsey et al., 2024)) are not captured.

- The 0.43 correlation between logit weight and steering effect leaves substantial variance unexplained.
- Manual judgment of steering success via regex newline-counting; an automated classifier (e.g., LLM judge) would be more rigorous.
- Auto-interp via Kimi K2 has documented weaknesses: it clustered 10+ distinct features all as "newline tokens." Discrimination required follow-up analysis.

6 Reproducibility

Full code, configuration, and result JSONs at https://github.com/OE-GOD/sae-pythia-160m. License MIT. Compute budget ∼$50 on Apple M-series via PyTorch MPS. SAE training: ∼50 minutes. Full analysis pipeline: ∼3 hours.

Acknowledgments

Auto-interp via Moonshot's Kimi K2. Built on the methodological foundations of (Bricken et al., 2023; Gao et al., 2024; Lieberum et al., 2024; Rajamanoharan et al., 2024).

References

Steven Bills, Nick Cammarata, Dan Mossing, Henk Tillman, Leo Gao, Gabriel Goh, Ilya Sutskever, Jan Leike, Jeffrey Wu, and William Saunders. Language models can explain neurons in language models. *OpenAI Blog*, 2023. URL https://openaipublic.blob.core.windows.net/neuron-explainer/paper/index.html.

Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, Robert Lasenby, Yifan Wu, Shauna Kravec, Nicholas Schiefer, Tim Maxwell, Nicholas Joseph, Alex Tamkin, Karina Nguyen, Brayden McLean, Josiah E Burke, Tristan Hume, Shan Carter, Tom Henighan, and Chris Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. URL https://transformer-circuits.pub/2023/monosemantic-features/.

Leo Gao, Tom Dupré la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever, Jan Leike, and Jeffrey Wu. Scaling and evaluating sparse autoencoders. *arXiv preprint arXiv:2406.04093*, 2024.

Stefan Heimersheim and Neel Nanda. How to use and interpret activation patching. *arXiv preprint arXiv:2404.15255*, 2024. URL https://arxiv.org/abs/2404.15255.

Tom Lieberum, Senthooran Rajamanoharan, Arthur Conmy, Lewis Smith, Nicolas Sonnerat, Vikrant Varma, János Kramár, Anca Dragan, Rohin Shah, and Neel Nanda. Gemma scope: Open sparse autoencoders everywhere all at once on gemma 2. *arXiv preprint arXiv:2408.05147*, 2024.

Jack Lindsey, Adly Templeton, Jonathan Marcus, Tom Conerly, Joshua Batson, and Chris Olah. Sparse crosscoders for cross-layer features and model diffing. *Transformer Circuits Thread*, 2024. URL https://transformer-circuits.pub/2024/crosscoders/.

Senthooran Rajamanoharan, Arthur Conmy, Lewis Smith, Tom Lieberum, Vikrant Varma, János Kramár, Rohin Shah, and Neel Nanda. Improving dictionary learning with gated sparse autoencoders. *arXiv preprint arXiv:2404.16014*, 2024.